

Guía Integral de Fundamentos de Programación Imperativa, Git y GitHub con SSH

Material de Apoyo para Estudiantes

Abstract

Esta guía académica cubre de manera secuencial la estructuración lógica mediante pseudocódigo, la implementación práctica utilizando el lenguaje Python en entornos de desarrollo eficientes, y la correcta gestión de versiones empleando Git y GitHub mediante el protocolo seguro SSH, previniendo errores de autenticación comunes en clase.

Índice

1	Introducción a la Programación Imperativa	2
2	El Fundamento Lógico: Pseudocódigo	2
3	La Implementación Práctica: Python	2
4	Gestión de Versiones con Git (Flujo Local)	2
4.1	Configuración Inicial de Identidad	2
4.2	El Ciclo de Trabajo Local	3
5	Conexión Segura con GitHub mediante SSH	3
5.1	Paso 1: Generación de Llaves	3
5.2	Paso 2: Copiado de la Llave Pública	3
5.3	Paso 3: Vinculación en GitHub y Repositorio	3
6	Solución de Problemas Frecuentes	4

1 Introducción a la Programación Imperativa

La programación imperativa describe la computación en términos de sentencias que cambian el estado de un programa. Especifica de forma explícita y secuencial los pasos que la computadora debe ejecutar.

Los pilares fundamentales incluyen:

- **Secuenciación:** Ejecución lineal de instrucciones.
- **Variables y Estados:** Espacios de memoria destinados a almacenar datos.
- **Estructuras de Decisión:** Ramificaciones lógicas según una condición.

2 El Fundamento Lógico: Pseudocódigo

A continuación, se presenta un algoritmo diseñado para evaluar el rendimiento académico de un estudiante asegurando la validación del umbral mínimo de aprobación:

```
INICIO
    // Declaración de variables de tipo flotante
    REAL calificacion

    // Entrada de datos del usuario
    ESCRIBIR "Ingrese la calificación final:"
    LEER calificacion

    // Estructura de decisión condicional
    SI calificacion >= 3.0 ENTONCES
        ESCRIBIR "El estudiante ha APROBADO."
    SINO
        ESCRIBIR "El estudiante ha REPROBADO."
    FIN SI
FIN
```

3 La Implementación Práctica: Python

La traducción directa del algoritmo anterior en Python se estructura de la siguiente manera:

```
# Entrada de datos: Conversión explícita a tipo flotante (float)
entrada_usuario = input("Ingrese la calificación del estudiante: ")
calificacion = float(entrada_usuario)

# Estructura de decisión condicional con indentación estricta
if calificacion >= 3.0:
    print("El estudiante ha APROBADO.")
else:
    print("El estudiante ha REPROBADO.")
```

4 Gestión de Versiones con Git (Flujo Local)

4.1 Configuración Inicial de Identidad

Antes de inicializar cualquier repositorio, es obligatorio configurar la identidad. Ejecute en la terminal:

```
$ git config --global user.name "Tu Nombre Completo"
$ git config --global user.email "tu_correo@universidad.edu.co"
```

4.2 El Ciclo de Trabajo Local

1. **Inicialización:** Crea la base de datos de Git.

```
$ git init
```

2. **Preparación de archivos (Staging):** Añade los archivos.

```
$ git add .
```

3. **Consolidación (Commit):** Captura la fotografía permanente.

```
$ git commit -m "Añadido script inicial para evaluar calificaciones"
```

5 Conexión Segura con GitHub mediante SSH

GitHub eliminó la autenticación mediante contraseñas por HTTPS. La solución es el protocolo SSH.

5.1 Paso 1: Generación de Llaves

```
$ ssh-keygen -t ed25519 -C "tu_correo@universidad.edu.co"
```

Presione **Enter** tres veces seguidas cuando se le hagan preguntas en la terminal para dejar la configuración por defecto y sin contraseña.

5.2 Paso 2: Copiado de la Llave Pública

```
$ cat ~/.ssh/id_ed25519.pub
```

Copie todo el bloque de texto impreso en pantalla.

5.3 Paso 3: Vinculación en GitHub y Repositorio

Pegue la llave en GitHub (Settings > SSH and GPG keys > New SSH key). Luego, vincule su código local:

```
$ git branch -M main
$ git remote add origin git@github.com:tu_usuario/nombre-del-repo.git
$ git push -u origin main
```

6 Solución de Problemas Frecuentes

Error en Pantalla	Causa Probable	Solución Técnica
Permission denied (publickey).	La llave pública no coincide.	Repita el Paso 3 asegurándose de copiar todo sin espacios extra.
fatal: remote origin already exists.	Ya se había vinculado una ruta remota.	Ejecute: <code>git remote remove origin</code> y vuelva a añadirlo.

Tabla 1: Matriz de diagnóstico y solución de errores comunes.